CONTENT AND LIFECYCLE IN JEE

Hands-On Lab Workbook

Platform: Java 11 LTS | Tomcat 9.0 | JAX-RS 2.1 (Jersey 2.x) | Maven 3.8+
Namespace: javax.ws.rs.* / javax.servlet.* (NOT jakarta.*)
Health endpoint: APPLICATION_XML (JAXB @XmlRootElement)
Config / JNDI / DB: TEXT_PLAIN (key=value lines)
Starter project: lifecycle-lab-starter.zip
Labs: 4 Labs — approx. 3 hours total

© Training Materials — All Rights Reserved

Lab Overview & Prerequisites

Package Namespace & Response Format — Important

All code uses javax.ws.rs.* (NOT jakarta.ws.rs.*) — correct for Java 11 / Tomcat 9.

Response format strategy for this module:

```
GET /api/health              → APPLICATION_XML  (JAXB @XmlRootElement bean)
Labs 1 & 2 (config, JNDI)    → TEXT_PLAIN  (key=value lines or raw strings)
Lab 3 (database / JDBC)      → TEXT_PLAIN  (CSV-format rows — JDBC data is tabular)
Lab 4 (security responses)   → TEXT_PLAIN  (error=... lines; status codes carry meaning)
HTML template (Lab 1)        → TEXT_HTML   (unchanged)
```

Note: Labs 1–3 deal with infrastructure (config, JNDI, SQL) rather than domain objects, so TEXT_PLAIN is the appropriate and honest format — no JAXB marshalling is involved.

Labs in This Workbook

Lab	Title	Covers	Section	Time
1	Reading Web Resources	@Context, ServletContext, init-params, WAR files	Sec 1	35 min
2	Finding JEE Components	JNDI, InitialContext, context.xml, @WebListener	Sec 2	35 min
3	Finding Databases	DataSource, connection pool, try-with-resources, transactions	Sec 3	50 min
4	Security Contexts	BearerTokenFilter, SecurityContext, @RolesAllowed, @Secured	Sec 4	50 min

Quick Start

1. Unzip lifecycle-lab-starter.zip and import as a Maven project.
2. Run: mvn clean package -DskipTests
3. Run: mvn test -Dtest=SmokeTest — all 8 baseline tests must PASS.
4. Deploy: cp target/lifecycle-lab.war $CATALINA_HOME/webapps/
5. Test: curl -H 'Accept: application/xml' http://localhost:8080/lifecycle-lab/api/health
6. Expected: <?xml...?>
<health><status>UP</status><service>LifecycleLab</service>...</health>

© Training Materials — All Rights Reserved

LAB 1 **Reading Web Resources** ⏱ 35 min
Section: Section 1

Objectives

- Inject ServletContext into a JAX-RS resource class using @Context
- Read application configuration from web.xml <context-param> entries
- Serve a static HTML file bundled inside the WAR
- Compare web-context resource access vs classpath resource access

Package Namespace & Response Format — Important

All code uses javax.ws.rs.* (NOT jakarta.ws.rs.*) — correct for Java 11 / Tomcat 9.

Response format strategy for this module:

GET /api/health → APPLICATION_XML (JAXB @XmlRootElement bean)
Labs 1 & 2 (config, JNDI) → TEXT_PLAIN (key=value lines or raw strings)
Lab 3 (database / JDBC) → TEXT_PLAIN (CSV-format rows — JDBC data is tabular)
Lab 4 (security responses) → TEXT_PLAIN (error=... lines; status codes carry meaning)
HTML template (Lab 1) → TEXT_HTML (unchanged)

Note: Labs 1–3 deal with infrastructure (config, JNDI, SQL) rather than domain objects, so TEXT_PLAIN is the appropriate and honest format — no JAXB marshalling is involved.

Task 1.1 — Inject ServletContext and Read Init-Params

Open ConfigResource.java. Add @Context injection and implement the two GET endpoints that read from web.xml. Responses are TEXT_PLAIN — configuration values are plain strings:

```
import javax.servlet.ServletContext;
import javax.ws.rs.*;
import javax.ws.rs.core.*;

@Path("/config")
public class ConfigResource {

    // Inject the Servlet container context
    @Context
    private ServletContext servletContext;

    // GET /api/config/version  → "2.4.1" as plain text
    @GET @Path("version")
    @Produces(MediaType.TEXT_PLAIN)
    public String version() {
        return servletContext.getInitParameter("app.version");
    }

    // GET /api/config  → all three params as key=value lines
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String allParams() {
        String version  = servletContext.getInitParameter("app.version");
        String payments =
servletContext.getInitParameter("feature.payments.enabled");
```

© Training Materials — All Rights Reserved

```
        String maxBytes =
servletContext.getInitParameter("upload.max.bytes");
        return "appVersion=" + version + "\n" +
               "paymentsEnabled=" + payments + "\n" +
               "maxUploadBytes=" + maxBytes + "\n";
    }
}

# Test:
curl http://localhost:8080/lifecycle-lab/api/config/version
# → 2.4.1

curl http://localhost:8080/lifecycle-lab/api/config
# → appVersion=2.4.1
#   paymentsEnabled=true
#   maxUploadBytes=10485760
```

Write your completed ConfigResource (version() and allParams()):

Task 1.2 — Serve a WAR File

Add a /api/config/template endpoint that reads and returns the HTML file at /WEB-INF/templates/welcome.html:

```
@GET @Path("template")
@Produces(MediaType.TEXT_HTML)
public Response template() throws IOException {
    InputStream is = servletContext
        .getResourceAsStream("/WEB-INF/templates/welcome.html");
    if (is == null) return Response.status(404).build();
    String html = new String(is.readAllBytes(), StandardCharsets.UTF_8);
    return Response.ok(html).build();
}

curl http://localhost:8080/lifecycle-lab/api/config/template
# → <html>...<h1>Welcome to the Lifecycle Lab</h1>...</html>
```

Task 1.3 — Classpath vs Web-Context Resources

Add /api/config/classpath that reads messages.properties from the classpath (WEB-INF/classes):

```
@GET @Path("classpath")
@Produces(MediaType.TEXT_PLAIN)
public Response classpathResource() throws IOException {
    // ClassLoader reads from WEB-INF/classes (the classpath root)
```

© Training Materials — All Rights Reserved

```
    InputStream is = getClass().getResourceAsStream("/messages.properties");
    if (is == null) return Response.status(404).build();
    return Response.ok(
        new String(is.readAllBytes(), StandardCharsets.UTF_8)
    ).build();
}

curl http://localhost:8080/lifecycle-lab/api/config/classpath
# → greeting=Hello from the Lifecycle Lab
#   support.email=support@example.com
```

Q1

What path would you use if the file was at src/main/resources/config/app.properties? Write the correct getResourceAsStream() call.

Your answer:

Q2

A colleague tries to read /WEB-INF/templates/welcome.html using getClass().getResourceAsStream("/WEB-INF/templates/welcome.html"). Will it work? Why or why not?

Your answer:

Lab 1 Checkpoint

- ✓ GET /api/config/version returns 2.4.1 as plain text
- ✓ GET /api/config returns all three init-params as key=value lines
- ✓ GET /api/config/template returns the welcome.html HTML content
- ✓ GET /api/config/classpath returns the messages.properties content

© Training Materials — All Rights Reserved

LAB 2

Finding JEE Components — JNDI

Section: Section 2

35 min

Objectives

- Read a JNDI-registered string value from Tomcat context.xml using InitialContext
- Implement a @WebListener that binds custom objects to JNDI at startup
- Verify that resource lookup works from within a JAX-RS resource class

Task 2.1 — Look Up a Named Value from JNDI

The context.xml already declares an Environment entry with name app.version and value 2.4.1. Open JndiResource.java and add a /api/jndi/version endpoint:

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import javax.naming.*;

@Path("/jndi")
@Produces(MediaType.TEXT_PLAIN)
public class JndiResource {

    @GET @Path("version")
    public Response jndiVersion() {
        try {
            Context ic = new InitialContext();
            // JNDI name for an Environment entry:
            // java:comp/env/ + the name attribute from context.xml
            String version = (String) ic.lookup("java:comp/env/app.version");
            return Response.ok(version).build();
        } catch (NameNotFoundException e) {
            return Response.status(404)
                .entity("JNDI name not found: " + e.getMessage()).build();
        } catch (NamingException e) {
            throw new InternalServerErrorException(e.getMessage(), e);
        }
    }
}

curl http://localhost:8080/lifecycle-lab/api/jndi/version
# → 2.4.1
```

Task 2.2 — Bind a Custom Object to JNDI at Startup

Open AppLifecycleListener.java. Implement contextInitialized() to bind a greeting string to JNDI. Then look it up from a new /api/jndi/greeting endpoint:

```
import javax.servlet.*;
import javax.servlet.annotation.WebListener;
import javax.naming.*;

@WebListener
public class AppLifecycleListener implements ServletContextListener {

    @Override
    public void contextInitialized(ServletContextEvent sce) {
```

© Training Materials — All Rights Reserved

```
        String version = sce.getServletContext()
                            .getInitParameter("app.version");
        sce.getServletContext().log(
            "[AppLifecycleListener] Starting — version: " + version);

        try {
            InitialContext ic = new InitialContext();
            // Ensure the subcontext exists
            try { ic.lookup("java:comp/env/app"); }
            catch (NameNotFoundException e) {
                ic.createSubcontext("java:comp/env/app");
            }
            // rebind overwrites if already exists (safe for hot redeploy)
            ic.rebind("java:comp/env/app/greeting",
                "Welcome to " + version);
            sce.getServletContext().log(
                "[AppLifecycleListener] JNDI resources bound.");
        } catch (NamingException e) {
            throw new RuntimeException("JNDI binding failed", e);
        }
    }

    @Override
    public void contextDestroyed(ServletContextEvent sce) {
        sce.getServletContext().log(
            "[AppLifecycleListener] Shutting down.");
    }
}

// JndiResource — add the greeting endpoint:
@GET @Path("greeting")
@Produces(MediaType.TEXT_PLAIN)
public Response jndiGreeting() {
    try {
        Context ic = new InitialContext();
        String greeting = (String) ic.lookup("java:comp/env/app/greeting");
        return Response.ok(greeting).build();
    } catch (NameNotFoundException e) {
        return Response.status(404)
            .entity("Greeting not bound — did AppLifecycleListener
run?").build();
    } catch (NamingException e) {
        throw new InternalServerErrorException(e.getMessage(), e);
    }
}

curl http://localhost:8080/lifecycle-lab/api/jndi/greeting
# → Welcome to 2.4.1
```

Write your completed AppLifecycleListener.contextInitialized() and JndiResource.jndiGreeting():

© Training Materials — All Rights Reserved

Q1

What would happen if you called ic.bind() instead of ic.rebind() on a second deployment without restarting Tomcat?

Your answer:

Lab 2 Checkpoint

- ✓ GET /api/jndi/version returns 2.4.1 (read from context.xml Environment entry)
- ✓ GET /api/jndi/greeting returns 'Welcome to 2.4.1' (bound at startup by listener)
- ✓ Tomcat log shows: [AppLifecycleListener] JNDI resources bound.

© Training Materials — All Rights Reserved

LAB 3 **Finding Databases — DataSource and JDBC** 50 min
Section: Section 3

Objectives

- Look up the H2 DataSource from JNDI and use it in a JAX-RS resource class
- Implement a connection-pool-safe list endpoint using try-with-resources
- Implement a POST endpoint that uses a manual transaction (setAutoCommit/commit/rollback)
- Deliberately cause a connection leak and observe the impact on the pool

Response format for Lab 3

The database endpoints return TEXT_PLAIN in CSV-style rows.
JDBC results are plain tabular data — not domain objects — so XML marshalling adds no value here. The plain text format makes results immediately readable with curl and avoids any dependency on a JSON/XML library in the data layer.

GET /api/db/products returns:
id,name,category,price
P001,Clean Code,books,39.99
P002,Effective Java,books,49.99
...

Task 3.1 — Look Up the DataSource and List Products

The context.xml already declares a jdbc/shopDB DataSource backed by H2. AppLifecycleListener creates the PRODUCTS table and seeds it. Open ProductResource.java and implement list():

```
import javax.servlet.ServletContext;
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import javax.naming.*;
import javax.sql.DataSource;
import java.sql.*;

@Path("/db")
@Produces(MediaType.TEXT_PLAIN)
public class ProductResource {

    private DataSource ds;

    @Context
    public void setServletContext(ServletContext ctx) {
        try {
            Context ic = new InitialContext();
            this.ds = (DataSource) ic.lookup("java:comp/env/jdbc/shopDB");
        } catch (NamingException e) {
            throw new RuntimeException("DataSource not found", e);
        }
    }

    // GET /api/db/products — returns CSV-format rows
    @GET @Path("products")
    public String list() {
```

© Training Materials — All Rights Reserved

```
        StringBuilder sb = new StringBuilder("id,name,category,price\n");
        String sql = "SELECT id, name, category, price FROM products ORDER BY
id";
        try (Connection c = ds.getConnection();
             PreparedStatement ps = c.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                sb.append(rs.getString("id")).append(",")
                  .append(rs.getString("name")).append(",")
                  .append(rs.getString("category")).append(",")
                  .append(rs.getBigDecimal("price")).append("\n");
            }
        } catch (SQLException e) {
            throw new InternalServerErrorException("DB error: " +
e.getMessage(), e);
        }
        return sb.toString();
    }
}

curl http://localhost:8080/lifecycle-lab/api/db/products
# → id,name,category,price
#   P001,Clean Code,books,39.99
#   P002,Effective Java,books,49.99
#   ...
```

Write your completed list() method:

Task 3.2 — POST with Manual Transaction

Implement the POST endpoint. Accept a plain-text body in the format id=PNEW,name=...,category=...,price=... and insert both product and audit log rows atomically:

```
// POST /api/db/products
// Body: id=PNEW,name=Design Patterns,category=books,price=44.99
@POST @Path("products")
@Consumes(MediaType.TEXT_PLAIN)
public Response create(String body) {
    // Parse key=value pairs from plain-text body
    Map<String,String> params = new LinkedHashMap<>();
    for (String pair : body.trim().split(",")) {
        String[] kv = pair.split("=", 2);
        if (kv.length == 2) params.put(kv[0].trim(), kv[1].trim());
    }
```

© Training Materials — All Rights Reserved

```
    String id       = params.get("id");
    String name     = params.get("name");
    String category = params.get("category");
    double price    = Double.parseDouble(params.getOrDefault("price","0"));

    try (Connection conn = ds.getConnection()) {
        conn.setAutoCommit(false);  // begin transaction
        try {
            // Insert product
            try (PreparedStatement ps = conn.prepareStatement(
                    "INSERT INTO products(id,name,category,price)
VALUES(?,?,?,?)")) {
                ps.setString(1,id); ps.setString(2,name);
                ps.setString(3,category); ps.setDouble(4,price);
                ps.executeUpdate();
            }
            // Insert audit log (atomic with product insert)
            try (PreparedStatement ps = conn.prepareStatement(
                    "INSERT INTO audit_log(action,entity_id) VALUES(?,?)")) {
                ps.setString(1,"CREATE"); ps.setString(2,id);
                ps.executeUpdate();
            }
            conn.commit();
            return Response.status(201)
                .entity("created=" + id + "\n").build();
        } catch (Exception e) {
            conn.rollback();  // undo BOTH inserts
            throw new InternalServerErrorException(e.getMessage(), e);
        }
    } catch (SQLException e) {
        throw new InternalServerErrorException(e.getMessage(), e);
    }
}

# Test:
curl -v -X POST http://localhost:8080/lifecycle-lab/api/db/products \
     -H 'Content-Type: text/plain' \
     -d 'id=PNEW,name=Design Patterns,category=books,price=44.99'
# → 201 Created   created=PNEW

# Confirm both tables updated:
curl http://localhost:8080/lifecycle-lab/api/db/products
curl http://localhost:8080/lifecycle-lab/api/db/audit
```

Write your completed create() method:

Q1

© Training Materials — All Rights Reserved

Describe what happens to data consistency if you call conn.commit() after the product insert but BEFORE the audit log insert crashes. Why does the all-or-nothing approach matter?

Your answer:

Q2 — Connection Leak Observation

Temporarily remove try-with-resources from list() (use old-style conn = ds.getConnection()).
Throw a RuntimeException BEFORE calling conn.close().
What does the Tomcat log show after 5 such requests (pool maxTotal=5)?
Restore try-with-resources and observe the difference.

Your observations (pool exhaustion behaviour):

Lab 3 Checkpoint

- ✓ GET /api/db/products returns CSV rows for the 6 seeded products
- ✓ POST /api/db/products creates a product AND an audit log entry atomically
- ✓ POST with a deliberate rollback (duplicate id) leaves BOTH tables unchanged
- ✓ GET /api/db/audit returns the audit log rows

© Training Materials — All Rights Reserved

LAB 4

Security Contexts — Authentication and Authorisation

Section: Section 4

50 min

Objectives

- Implement a Bearer token authentication filter using ContainerRequestFilter
- Install a custom SecurityContext so resource methods can call isUserInRole()
- Apply @Secured name-binding to protect only specific endpoints
- Add @RolesAllowed to require specific roles on protected endpoints
- Verify that 401 is returned for missing tokens and 403 for insufficient roles

Security error response format

BearerTokenFilter.abort() returns TEXT_PLAIN error messages.
The HTTP status code carries the semantic meaning (401 vs 403).
Format: error=<message>
status=<code>

This is consistent with the plain-text convention used throughout this module.

Task 4.1 — Complete BearerTokenFilter

Open BearerTokenFilter.java. The skeleton has @Provider and @Priority(Priorities.AUTHENTICATION). Implement the filter() method:

```
import javax.annotation.Priority;
import javax.ws.rs.Priorities;
import javax.ws.rs.container.ContainerRequestContext;
import javax.ws.rs.container.ContainerRequestFilter;
import javax.ws.rs.core.*;
import javax.ws.rs.ext.Provider;

@Provider
@Secured                              // name-binding: only @Secured
endpoints
@Priority(Priorities.AUTHENTICATION)  // runs before resource methods
public class BearerTokenFilter implements ContainerRequestFilter {

    private static final String BEARER_PREFIX = "Bearer ";

    @Override
    public void filter(ContainerRequestContext ctx) {
        String auth = ctx.getHeaderString(HttpHeaders.AUTHORIZATION);

        if (auth == null || !auth.startsWith(BEARER_PREFIX)) {
            abort(ctx, 401, "Missing or malformed Authorization header");
            return;
        }
        String token = auth.substring(BEARER_PREFIX.length()).trim();
        UserPrincipal principal = TokenStore.validate(token);

        if (principal == null) {
            abort(ctx, 401, "Invalid or expired token");
```

© Training Materials — All Rights Reserved

```
            return;
        }
        ctx.setSecurityContext(new BearerSecurityContext(
            principal, ctx.getSecurityContext().isSecure()));
    }

    private void abort(ContainerRequestContext ctx, int status, String
message) {
        ctx.abortWith(Response.status(status)
            .entity("error=" + message + "\nstatus=" + status + "\n")
            .type(MediaType.TEXT_PLAIN)
            .header(HttpHeaders.WWW_AUTHENTICATE,
                    "Bearer realm=\"lifecycle-lab\"")
            .build());
    }
}
```

Write your completed BearerTokenFilter.filter() method:

Task 4.2 — Apply @Secured to SecuredOrderResource

Open SecuredOrderResource.java. Add @Secured to the create() method. Implement create() to read the principal from SecurityContext:

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;

@Path("/secured/orders")
@Produces(MediaType.TEXT_PLAIN)
public class SecuredOrderResource {

    private static final List<String> ORDERS = new ArrayList<>();

    // Public — no token needed
    @GET
    public String list() {
        return ORDERS.isEmpty() ? "orders=(empty)\n"
            : "orders:\n" + String.join("\n", ORDERS) + "\n";
    }

    // Protected — requires valid Bearer token
    @POST
    @Secured   // ← add this annotation
    @Consumes(MediaType.TEXT_PLAIN)
    public Response create(String order, @Context SecurityContext sc) {
        String user = sc.getUserPrincipal().getName();
        ORDERS.add(user + ": " + order.trim());
```

© Training Materials — All Rights Reserved

```
        return Response.status(201)
            .entity("created=true\ncreatedBy=" + user + "\n")
            .build();
    }
}

# Test all scenarios:
# 1. GET (public) — no token needed
curl http://localhost:8080/lifecycle-lab/api/secured/orders
# → orders=(empty)

# 2. POST with missing token → 401
curl -v -X POST http://localhost:8080/lifecycle-lab/api/secured/orders \
     -H 'Content-Type: text/plain' -d 'book order'
# → 401  error=Missing or malformed Authorization header

# 3. POST with invalid token → 401
curl -v -X POST http://localhost:8080/lifecycle-lab/api/secured/orders \
     -H 'Authorization: Bearer fake-token' \
     -H 'Content-Type: text/plain' -d 'book order'
# → 401  error=Invalid or expired token

# 4. POST with valid user token → 201
curl -v -X POST http://localhost:8080/lifecycle-lab/api/secured/orders \
     -H 'Authorization: Bearer user-token' \
     -H 'Content-Type: text/plain' -d 'Clean Code x1'
# → 201  created=true  createdBy=alice
```

Record the HTTP status code and response body for each of the four curl calls:

Task 4.3 — Add @RolesAllowed to AdminResource

Open AdminResource.java. Add the role annotations to the correct methods. Register RolesAllowedDynamicFeature in LifecycleApplication for @RolesAllowed to take effect:

```
import javax.annotation.security.DenyAll;
import javax.annotation.security.PermitAll;
import javax.annotation.security.RolesAllowed;
import javax.ws.rs.*;
import javax.ws.rs.core.*;

@Path("/admin")
@Secured                    // class-level: ALL methods require authentication
@Produces(MediaType.TEXT_PLAIN)
public class AdminResource {

    @GET @Path("stats")
    @RolesAllowed("admin")   // only admin role
    public Response stats(@Context SecurityContext sc) {
        return Response.ok(
            "calledBy=" + sc.getUserPrincipal().getName() +
            "\nproductCount=6\n").build();
```

© Training Materials — All Rights Reserved

```
    }

    @GET @Path("ping")
    @PermitAll               // overrides @Secured — no auth needed
    public Response ping() {
        return Response.ok("status=pong\n").build();
    }

    @GET @Path("locked")
    @DenyAll                 // always 403
    public Response locked() {
        return Response.ok("status=accessible\n").build();
    }

    @GET @Path("reports")
    @RolesAllowed({"admin","manager"})  // either role
    public Response reports(@Context SecurityContext sc) {
        return Response.ok(
            "calledBy=" + sc.getUserPrincipal().getName() +
            "\nreports=Q1-2024,Q2-2024\n").build();
    }
}

// Register in LifecycleApplication.java for @RolesAllowed to work:
@Override
public Set<Class<?>> getClasses() {
    return Set.of(
        resource.HealthResource.class,
        resource.ConfigResource.class,
        resource.JndiResource.class,
        resource.ProductResource.class,
        resource.SecuredOrderResource.class,
        resource.AdminResource.class,
        filter.BearerTokenFilter.class,
        org.glassfish.jersey.server.filter.RolesAllowedDynamicFeature.class
    );
}

# alice (role: user) → 403 on stats
curl -v -H 'Authorization: Bearer user-token' \
     http://localhost:8080/lifecycle-lab/api/admin/stats

# bob (role: admin) → 200 on stats
curl -v -H 'Authorization: Bearer admin-token' \
     http://localhost:8080/lifecycle-lab/api/admin/stats

# ping — no token needed (@PermitAll)
curl http://localhost:8080/lifecycle-lab/api/admin/ping
# → status=pong

# locked — always 403 (@DenyAll)
curl -v -H 'Authorization: Bearer admin-token' \
     http://localhost:8080/lifecycle-lab/api/admin/locked
```

Record the status codes for each curl call and explain why each is correct:

© Training Materials — All Rights Reserved

Q1

A class has @Secured at class level and @PermitAll on one method. Does @PermitAll override @Secured for that method? Test it and explain what Jersey does with the two annotations.

Your answer:

Q2

What is the difference between returning 401 and 403? A request arrives with a completely absent Authorization header. Which status should you return and why?

Your answer:

Lab 4 Checkpoint

- ✓ GET /api/secured/orders (no token) returns 200 — public endpoint
- ✓ POST /api/secured/orders (no token) returns 401 with plain-text error
- ✓ POST /api/secured/orders (invalid token) returns 401
- ✓ POST /api/secured/orders (user-token) returns 201 createdBy=alice
- ✓ GET /api/admin/stats with user-token returns 403
- ✓ GET /api/admin/stats with admin-token returns 200
- ✓ GET /api/admin/ping (no token) returns 200 — @PermitAll overrides @Secured
- ✓ GET /api/admin/locked (any token) returns 403 — @DenyAll

© Training Materials — All Rights Reserved

Model Answers — Key Code

Reference implementations. Consult only after attempting each lab.

Lab 1 — ConfigResource

```
@GET @Path("version") @Produces(MediaType.TEXT_PLAIN)
public String version() {
    return servletContext.getInitParameter("app.version");
}

@GET @Produces(MediaType.TEXT_PLAIN)
public String allParams() {
    return "appVersion=" + servletContext.getInitParameter("app.version") +
"\n" +
           "paymentsEnabled=" +
servletContext.getInitParameter("feature.payments.enabled") + "\n" +
           "maxUploadBytes=" +
servletContext.getInitParameter("upload.max.bytes") + "\n";
}
```

Lab 2 — AppLifecycleListener JNDI binding

```
ic.rebind("java:comp/env/app/greeting",
    "Welcome to " + sce.getServletContext().getInitParameter("app.version"));
```

Lab 3 — ProductResource.list()

```
@GET @Path("products") @Produces(MediaType.TEXT_PLAIN)
public String list() {
    StringBuilder sb = new StringBuilder("id,name,category,price\n");
    try (Connection c = ds.getConnection();
         PreparedStatement ps = c.prepareStatement(
             "SELECT id,name,category,price FROM products ORDER BY id");
         ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            sb.append(rs.getString("id")).append(",")
              .append(rs.getString("name")).append(",")
              .append(rs.getString("category")).append(",")
              .append(rs.getBigDecimal("price")).append("\n");
        }
    } catch (SQLException e) {
        throw new InternalServerErrorException("DB error: " + e.getMessage(),
e);
    }
    return sb.toString();
}
```

Lab 4 — BearerTokenFilter.filter()

```
@Override
public void filter(ContainerRequestContext ctx) {
```

© Training Materials — All Rights Reserved

```
    String auth = ctx.getHeaderString(HttpHeaders.AUTHORIZATION);
    if (auth == null || !auth.startsWith("Bearer ")) {
        abort(ctx, 401, "Missing or malformed Authorization header"); return;
    }
    String token = auth.substring("Bearer ".length()).trim();
    UserPrincipal principal = TokenStore.validate(token);
    if (principal == null) {
        abort(ctx, 401, "Invalid or expired token"); return;
    }
    ctx.setSecurityContext(new BearerSecurityContext(
        principal, ctx.getSecurityContext().isSecure()));
}
```

© Training Materials — All Rights Reserved